

Course Title : Neural Networks and Deep Learning

Course Code:AMD504A

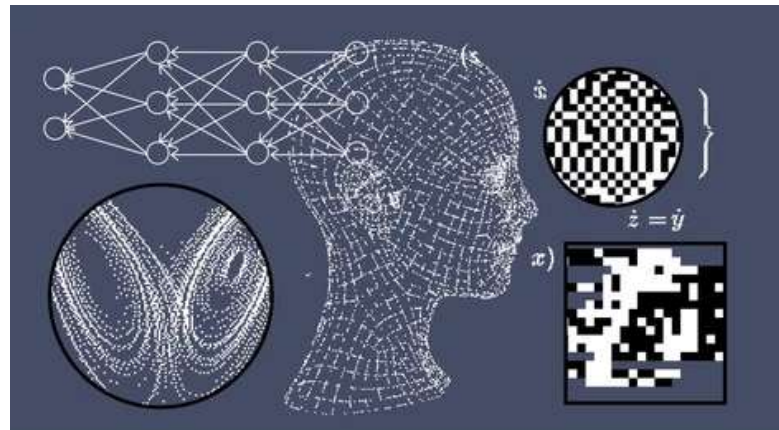
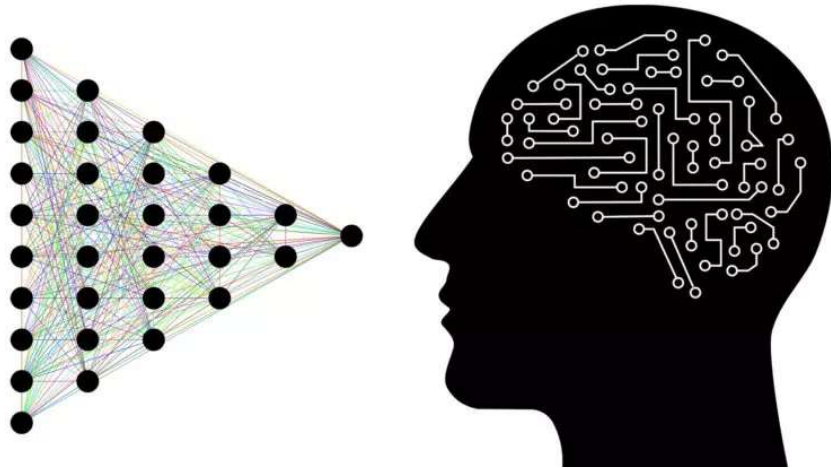
Course Teacher:

Dr Boppuru Rudra Prathap

boppururudraprathap.cs.et@msruas.ac.in



Neural Networks and Deep Learning

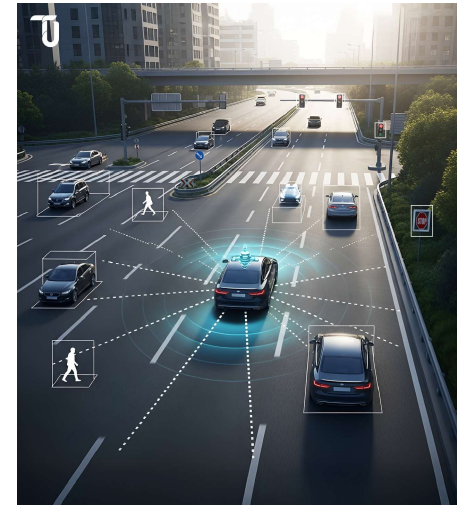


shutterstock.com · 2707402769



Why Neural Networks & Deep Learning?

- Deep learning is currently an extremely **active area of research** and is the foundation for modern machine learning.
- It allows for the modeling of complex tasks that traditional algorithms struggle with, such as **image recognition and natural language processing**.
- Knowledge in this field is essential for roles in **self-driving car development, healthcare AI, ChatGPT & NLP and automated fraud detection**.



Real-World Problem

- How does Netflix recommend movies?
- How does Google Translate work?
- How does Face Unlock identify you?

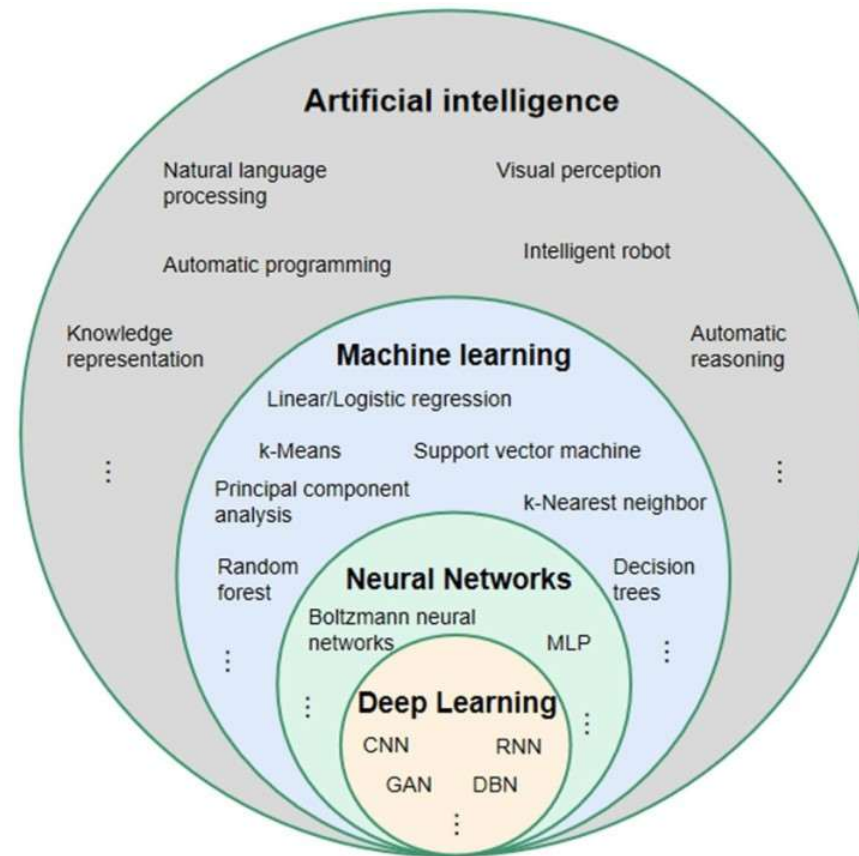


Unit Flow

- NN → CNN → RNN → GAN → RL → Advanced AI
- This course builds layer by layer understanding

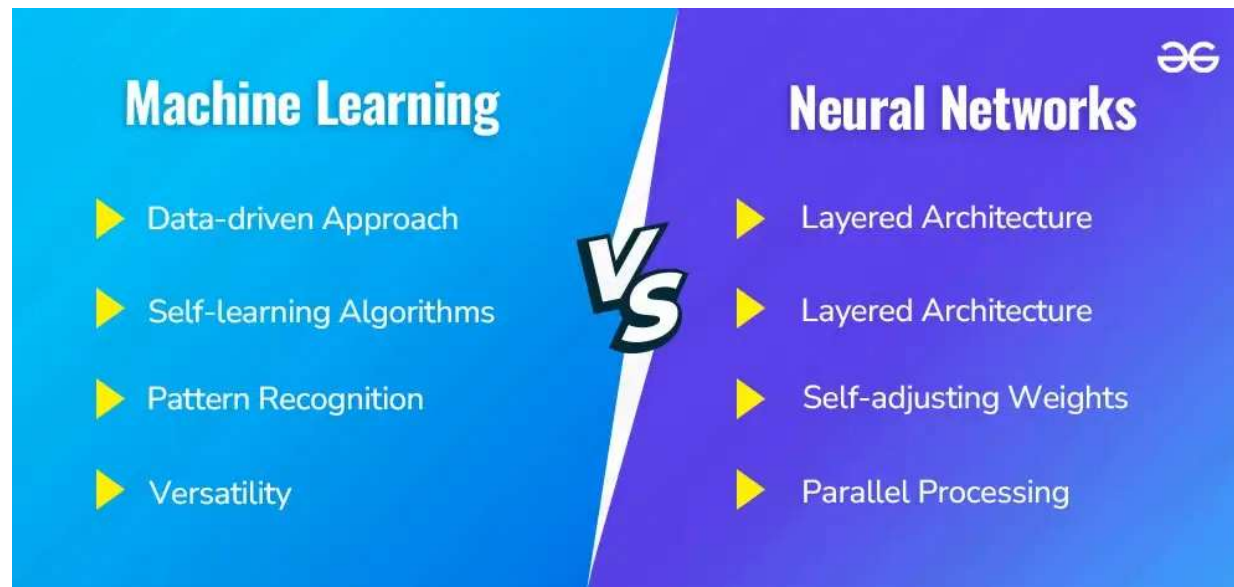


AI-ML-NN-DL Overview



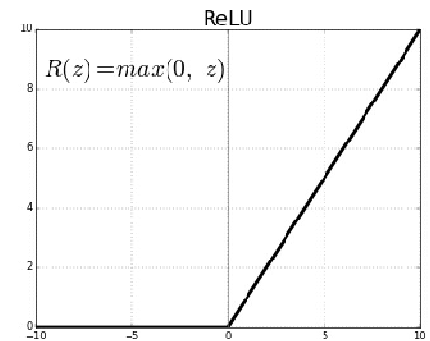
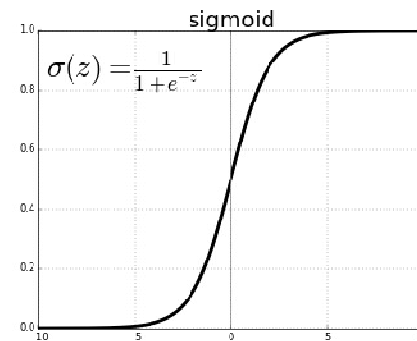
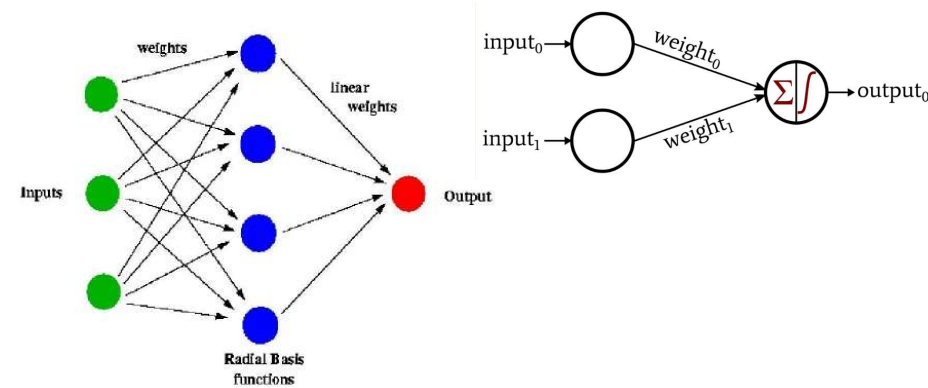
Why Neural Networks?

- Traditional Machine Learning struggles with complex, non-linear data
- Neural Networks can automatically learn patterns and features.



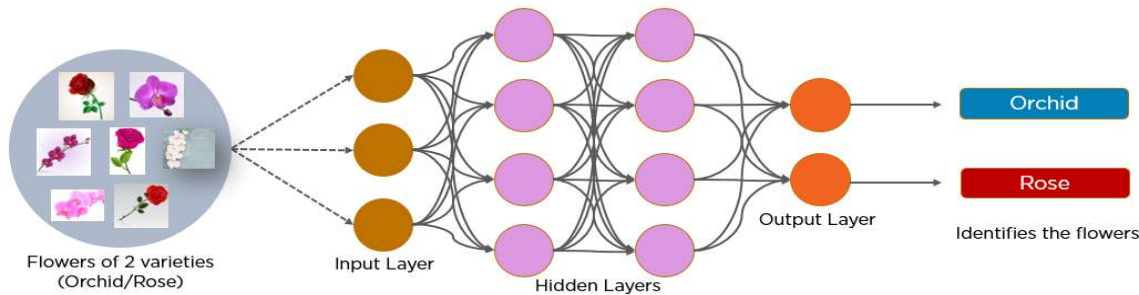
Unit 1 - Introduction to Neural Networks

- **Fundamentals:** Understanding the basic structure: Input Layer, Hidden Layers, and Output Layer.
- **Biological Inspiration:** How mathematical models mimic the way biological neurons process signals.
- **Key Algorithms:**
 - **Activation Functions:** Sigmoid, tanh, and ReLU.
 - Example: Decision boundaries
 - **Training:** Gradient Descent and Backpropagation for adjusting network weights for Loss minimization
- **Regularization:** Techniques like Dropout and L1/L2 regularization to prevent overfitting.



Unit 1: Introduction to Neural Networks

- Introduction to the fundamentals of neural networks, historical background, and applications in various fields, understanding the basic structure and components of a neural network: input layer, hidden layers, and output layer, multi-layer perceptron (MLP), deep feedforward networks ,exploring different activation functions: sigmoid, tanh, ReLU, introduction to gradient descent and backpropagation algorithms for training neural networks, understanding weight initialization and its influence on network convergence and performance, exploring regularization techniques: dropout, L1/L2 regularization



Neural network that identifies two types of flowers: Orchid and Rose.



Introduction Neural Networks

- Neural networks are machine learning models that mimic the complex functions of the human brain.
- These models consist of interconnected nodes or neurons that process data, learn patterns and enable tasks such as pattern recognition and decision-making.

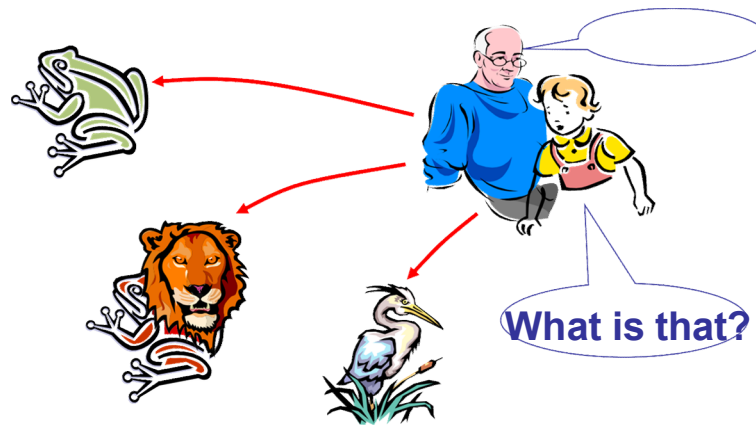
Used for:

- Image recognition (e.g., facial recognition in smartphones).
- Speech processing (e.g., virtual assistants like Siri or Alexa).
- Predictive analytics (e.g., weather forecasting).



Introduction Neural Networks

- **NNs learn relationship between cause and effect or organize large volumes of data into orderly and informative patterns.**



Neural networks to the rescue...

- **Neural network:** *information processing paradigm inspired by biological nervous systems, such as our brain*
- Structure: large number of highly interconnected processing elements (*neurons*) working together
- Like people, they learn *from experience* (by example)



Definition of ANN

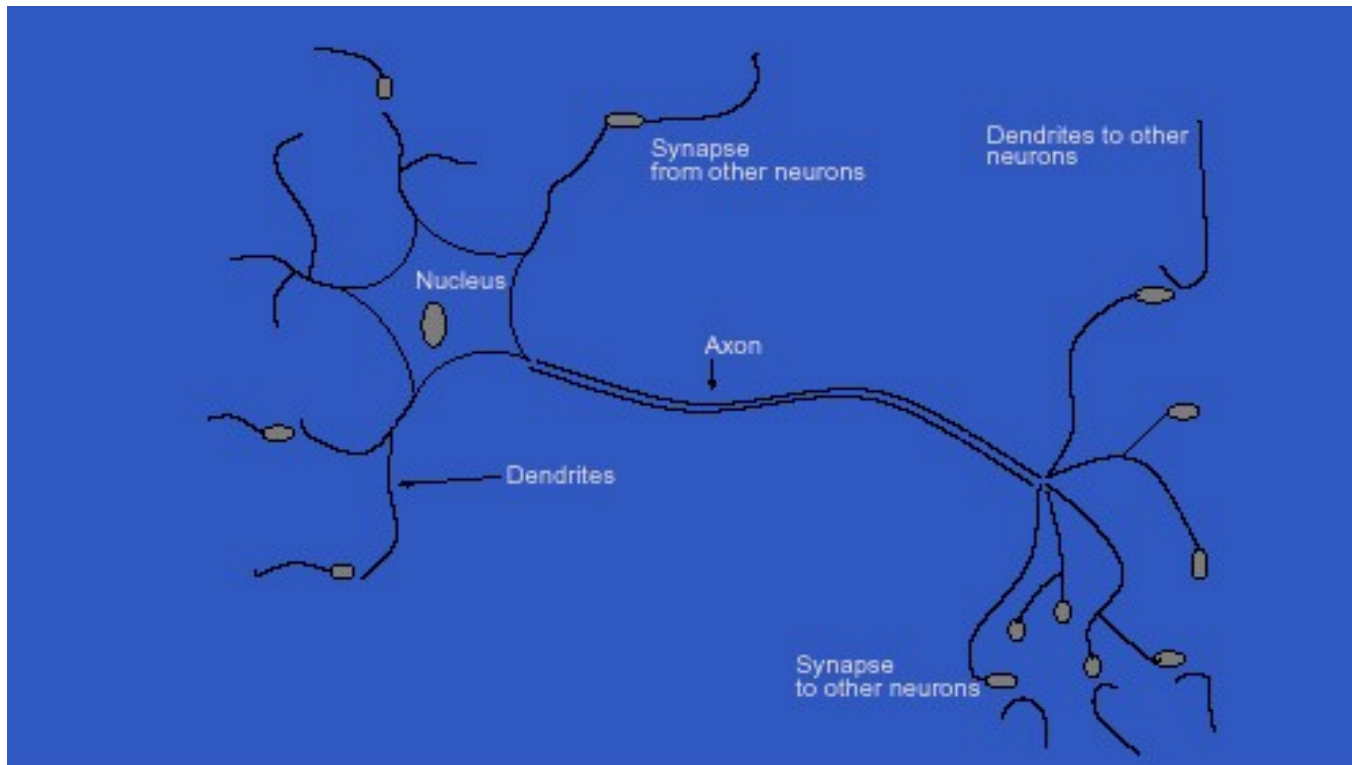
“Data processing system consisting of a large number of simple, highly interconnected processing elements (artificial neurons) in an architecture inspired by the structure of the cerebral cortex of the brain”

(Tsoukalas & Uhrig, 1997).

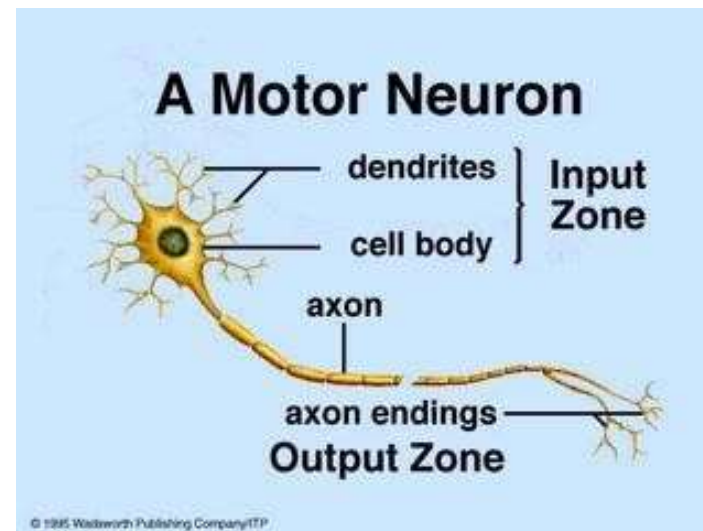
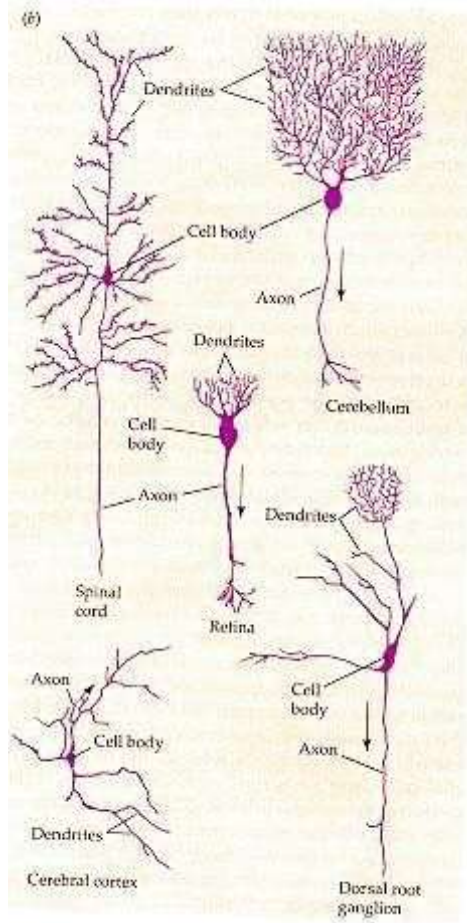


Inspiration from Neurobiology

Human Biological Neuron



Biological Neural Networks

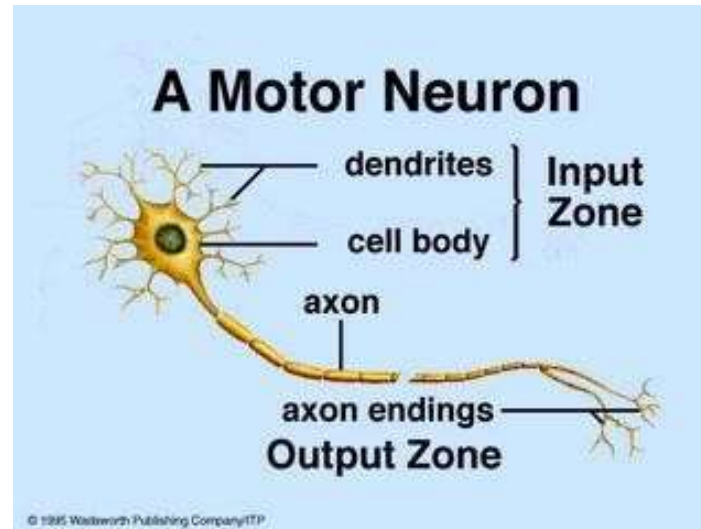


Biological neuron



Biological Neural Networks

- A biological neuron has three types of main components; dendrites, soma (or cell body) and axon.
- Dendrites receives signals from other neurons.
- The soma, sums the incoming signals. When sufficient input is received, the cell fires; that is it transmit a signal over its axon to other cells.



Artificial Neurons

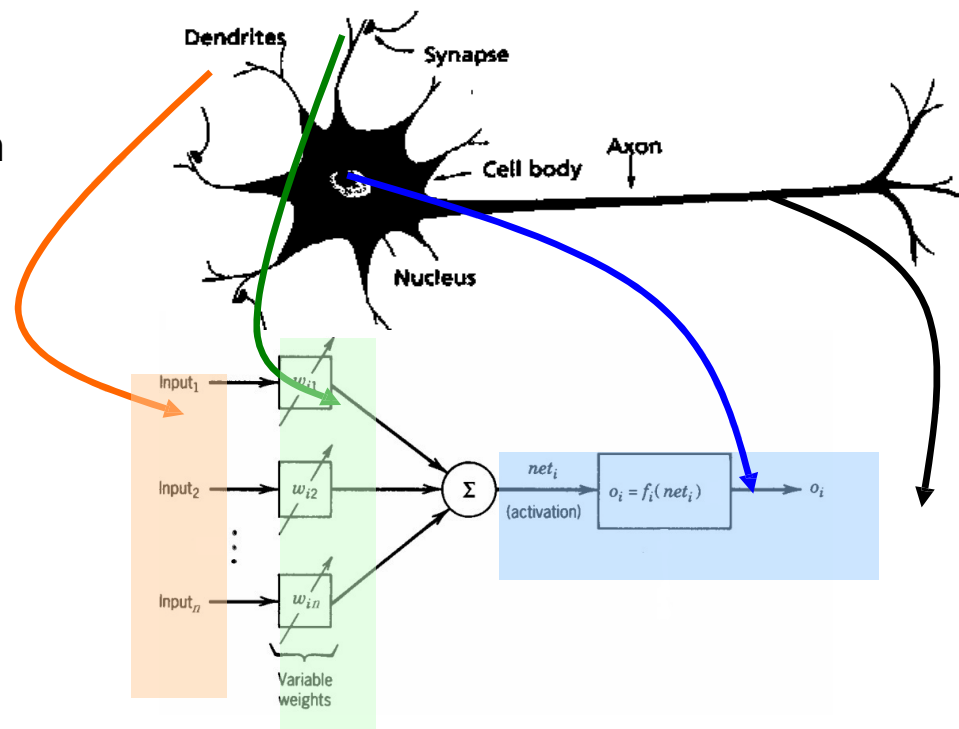
- ANN is an information processing system that has certain performance characteristics in common with biological nets.
- Several key features of the processing elements of ANN are suggested by the properties of biological neurons:
 1. The processing element receives many signals.
 2. Signals may be modified by a weight at the receiving synapse.
 3. The processing element sums the weighted inputs.
 4. Under appropriate circumstances (sufficient input), the neuron transmits a single output.
 5. The output from a particular neuron may go to many other neurons.



Artificial Neurons

- From experience: examples / training data
- Strength of connection between the neurons is stored as a weight-value for the specific connection.
- Learning the solution to a problem = changing the connection weights

A physical neuron



An artificial neuron



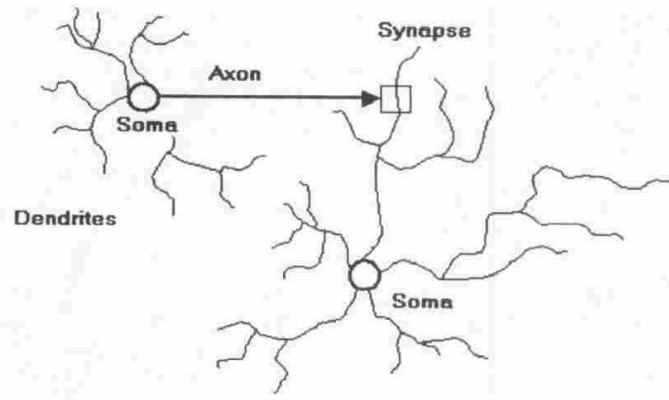
Artificial Neurons

■ ANNs have been developed as generalizations of mathematical models of neural biology, based on the assumptions that:

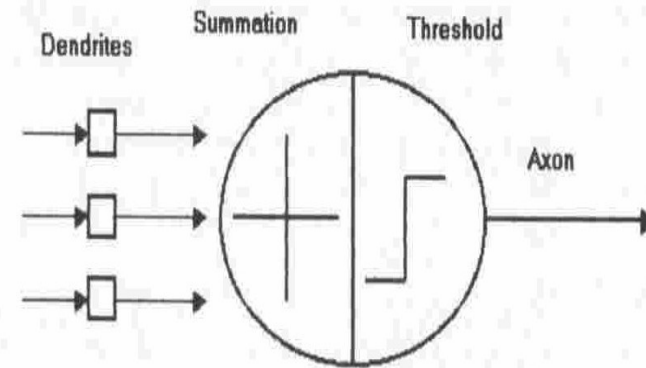
1. Information processing occurs at many simple elements called neurons.
2. Signals are passed between neurons over connection links.
3. Each connection link has an associated weight, which, in typical neural net, multiplies the signal transmitted.
4. Each neuron applies an activation function to its net input to determine its output signal.



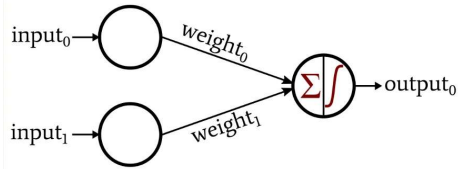
Artificial Neuron



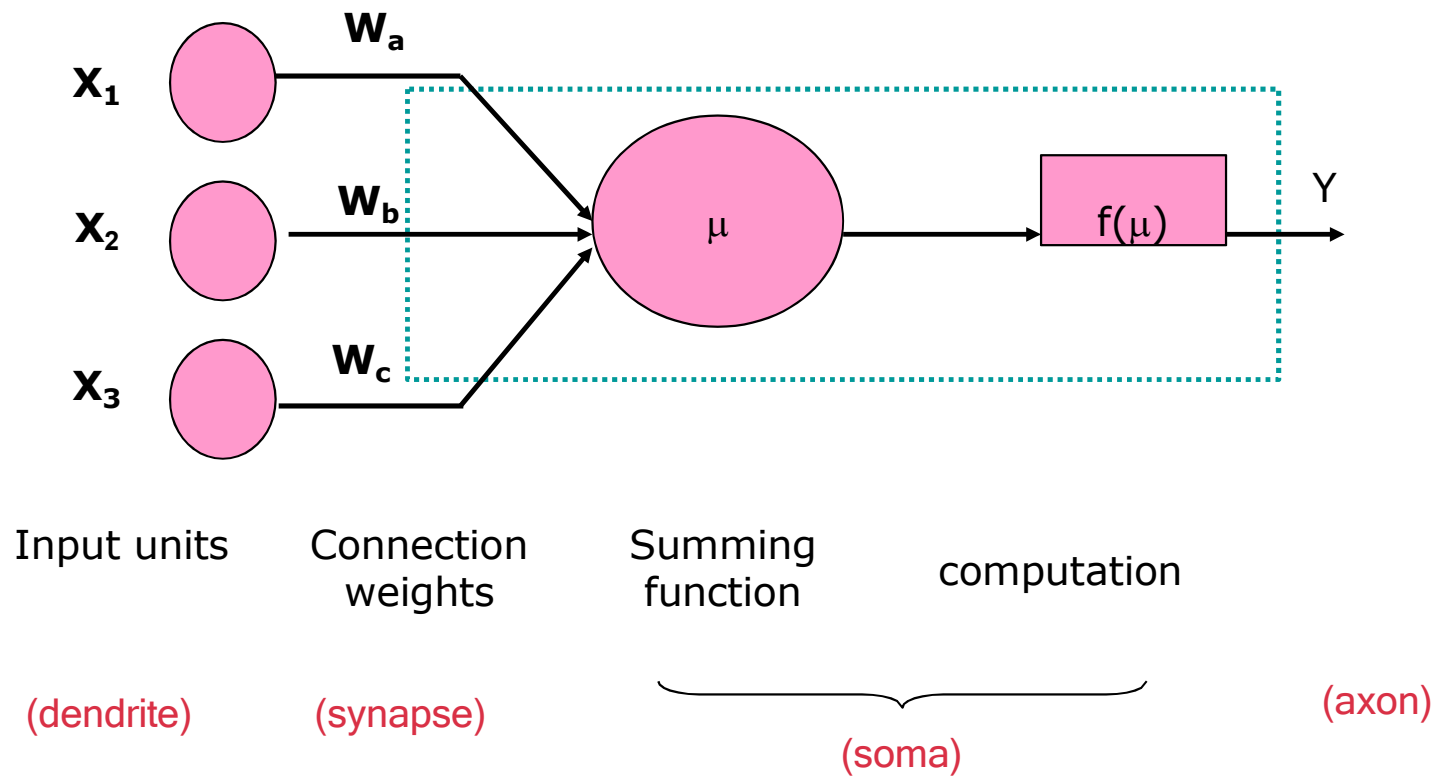
Four basic components of a human biological neuron



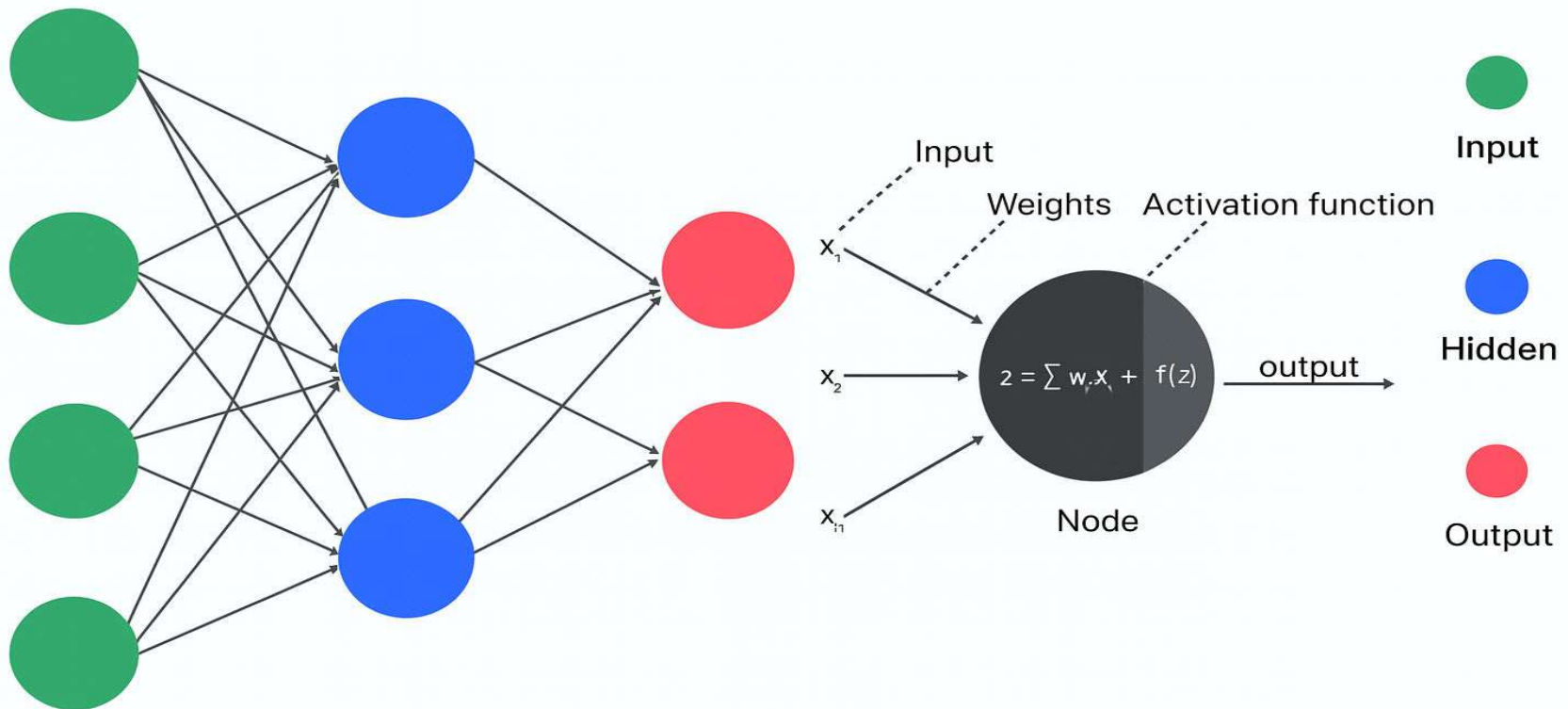
The components of a basic artificial neuron



Model Of A Neuron



Neural Network Architecture



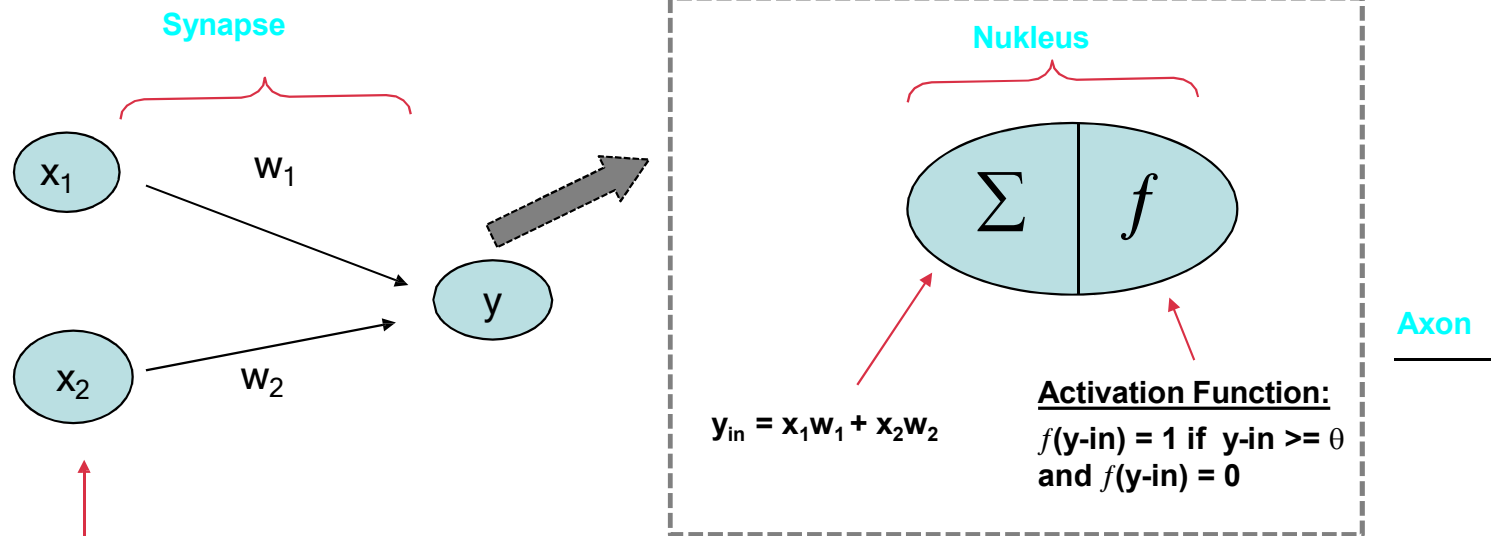
- A neural net consists of a large number of simple processing elements called neurons, units, cells or nodes.
- Each neuron is connected to other neurons by means of directed communication links, each with associated weight.
- The weight represent information being used by the net to solve a problem.



- Each neuron has an internal state, called its activation or activity level, which is a function of the inputs it has received. Typically, a neuron sends its activation as a signal to several other neurons.
 - It is important to note that a neuron can send only one signal at a time, although that signal is broadcast to several other neurons.
 - Neural networks are configured for a specific application, such as pattern recognition or data classification, through a learning process
 - In a biological system, learning involves adjustments to the synaptic connections between neurons
- ➔ same for artificial neural networks (ANNs)



Artificial Neural Network



Dendrite

- A neuron receives input, determines the strength or the weight of the input, calculates the total weighted input, and compares the total weighted with a value (threshold)
- The value is in the range of 0 and 1
- If the total weighted input greater than or equal the threshold value, the neuron will produce the output, and if the total weighted input less than the threshold value, no output will be produced



- **Neural networks are capable of learning and identifying patterns directly from data without pre-defined rules.**

These networks are built from several key components:

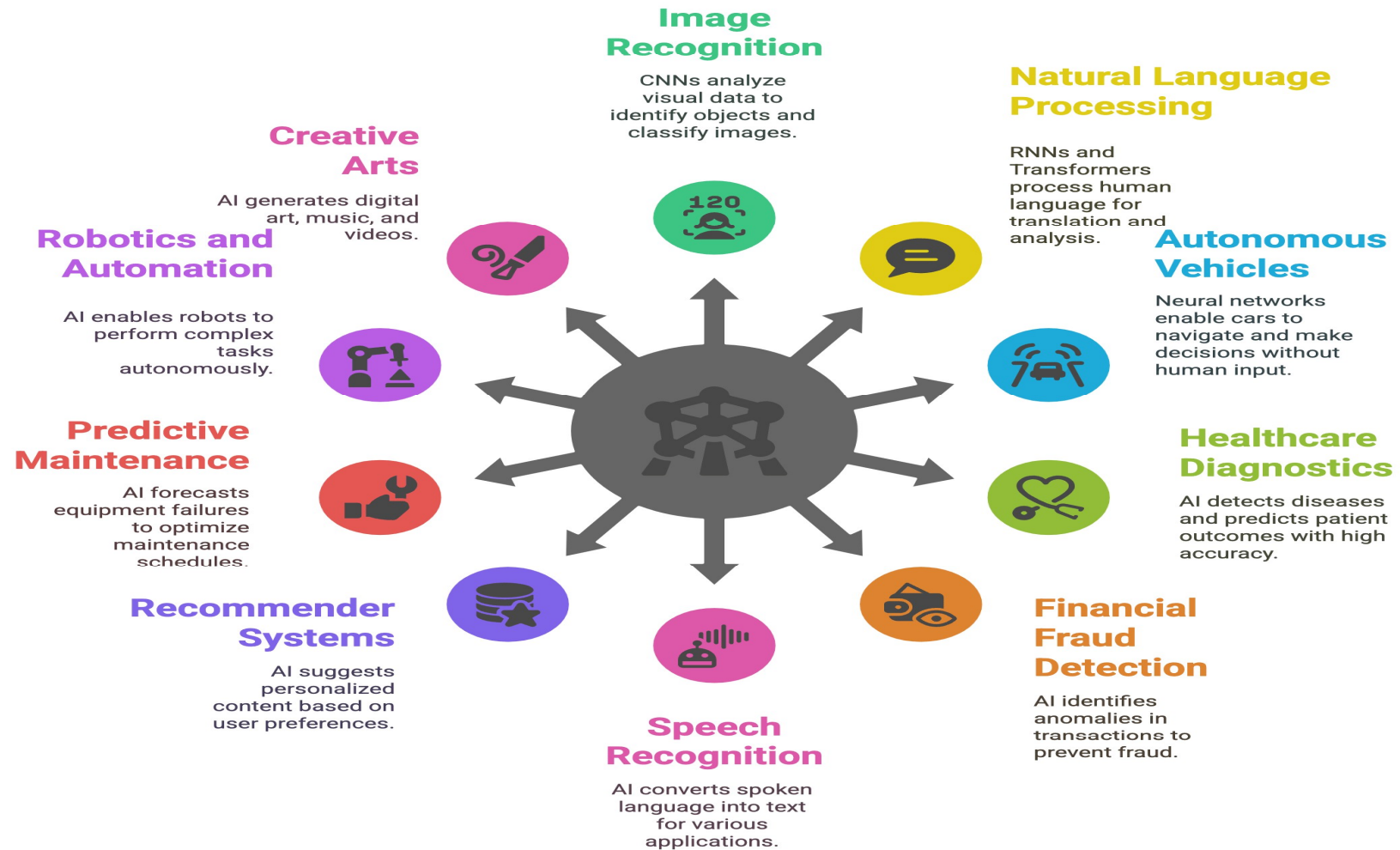
- **Neurons:** The basic units that receive inputs, each neuron is governed by a threshold and an activation function.
- **Connections:** Links between neurons that carry information, regulated by weights and biases.
- **Weights and Biases:** These parameters determine the strength and influence of connections.
- **Propagation Functions:** Mechanisms that help process and transfer data across layers of neurons.
- **Learning Rule:** The method that adjusts weights and biases over time to improve accuracy.



Applications of Neural Networks



Neural Networks Applications



Applications of Artificial Neural Networks

- **Social Media:** ANNs help social media platforms suggest friends and relevant content by analyzing user profiles, interests and interactions. They also assist in targeted advertising which ensures users to see ads tailored to their preferences.
- **Marketing and Sales:** E-commerce sites like Amazon use ANNs to recommend products based on browsing history. They also personalize offers, predict customer behavior and segment customers for more effective marketing campaigns.
- **Healthcare:** ANNs are used in medical imaging for detecting diseases like cancer and they assist in diagnosing conditions with accuracy similar to doctors. Additionally, they predict health risks and recommend personalized treatment plans.
- **Personal Assistants:** Virtual assistants like Siri and Alexa use ANNs to process natural language, understand voice commands and respond accordingly. They help manage tasks like setting reminders helps in making calls and answering queries.
- **Customer Support:** ANNs power chatbots and automated customer service systems that analyze customer queries and provide accurate responses helps in improving efficiency in handling customer inquiries.
- **Finance:** In the financial industry, they are used for fraud detection, credit scoring and predicting market trends by analyzing large sets of transaction data and spotting anomalies.



Challenges in Artificial Neural Networks

- **Data Dependency:** ANNs require large amounts of high-quality data to train effectively. Gathering and cleaning sufficient data can be time-consuming, expensive and often impractical especially in industries with limited access to quality data.
- **Computational Power:** Training deep neural networks with many layers, demands significant computational resources. High-performance hardware (e.g GPUs) is often required which makes it expensive and resource-intensive.
- **Overfitting:** It can easily overfit to the training data which means they perform well on the training set but poorly on new, unseen data. This challenge arises when the model learns to memorize rather than generalize, reducing its real-world applicability.
- **Interpretability:** They are often referred to as "black boxes." It is difficult to understand how they make decisions which is a problem in fields like healthcare and finance where explainability and transparency are important.
- **Training Time:** Training ANNs can take a long time, especially for deep learning models with many layers and vast datasets. This lengthy training process can delay the deployment of models and hinder their use in time-sensitive applications.



History

- Neural networks evolved through cycles of rise, fall, and revival.
- Neural networks have undergone significant evolution since their inception in the mid-20th century. Here's a concise timeline of the major developments in the field:
- **1940s-1950s:** The concept of neural networks began with McCulloch and Pitts' introduction of the first mathematical model for **artificial neurons**. However, the lack of computational power during that time posed significant challenges to further advancements.
- **1960s-1970s:** Frank Rosenblatt's worked on perceptrons. [Perceptrons](#) are simple single-layer networks that can solve linearly separable problems, but cannot perform complex tasks.
- **1980s:** The development of **backpropagation** by Rumelhart, Hinton, and Williams revolutionized neural networks by enabling the training of multi-layer networks. This period also saw the rise of connectionism, emphasizing learning through interconnected nodes.
- **1990s:** Neural networks experienced a surge in popularity with applications across image recognition, finance, and more. However, this growth was tempered by a period known as the **"AI winter,"** during which high computational costs and unrealistic expectations dampened progress.
- **2000s:** A resurgence was triggered by the availability of larger datasets, advances in computational power, and innovative network architectures. Deep learning, utilizing multiple layers, proved highly effective across various domains.
- **2010s:** The landscape of machine learning has been dominated by deep learning with **CNNs** (Convolutional Neural Networks) excelling in image classification and **RNNs** (Recurrent Neural Networks) , **LSTMs**, and **GRUs** gaining traction in sequence-based tasks like language modeling and speech recognition.
- **2017: Transformer models**, introduced by Vaswani et al. in "Attention is All You Need," revolutionized NLP by using a self-attention mechanism for parallel processing, improving efficiency. Models like **BERT**, **GPT**, and **T5** set new benchmarks in machine translation and text generation.



History And Evolution Of Neural Networks

01

1943

First Artificial Neuron Model By McCulloch And Pitts

02

1958

Rosenblatt Develops The Perceptron

03

1970

AI Winter Due To Perceptron Limitations

04

1986

Hinton Introduces Backpropagation

05

2012

CNN Wins ImageNet Competition, Boosting Deep Learning

06

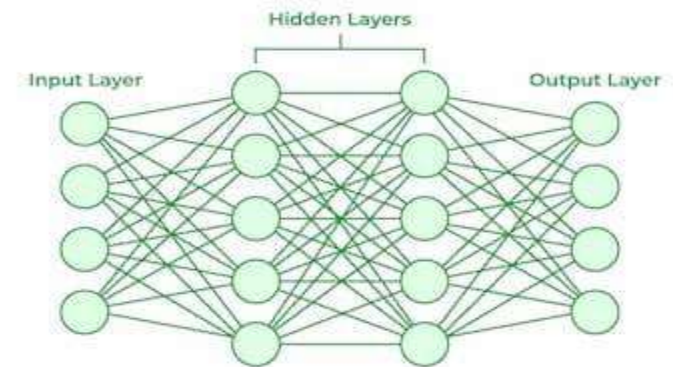
2020

Transformers Advance AI In NLP And Generative Models



Layers in Neural Network Architecture

- **Input Layer:** This is where the network receives its input data. Each input neuron in the layer corresponds to a feature in the input data.
- **Hidden Layers:** These layers perform most of the computational heavy lifting. A neural network can have one or multiple hidden layers. Each layer consists of units (neurons) that transform the inputs into something that the output layer can use.
- **Output Layer:** The final layer produces the output of the model. The format of these outputs varies depending on the specific task (e.g., classification, regression).



Working of Neural Networks

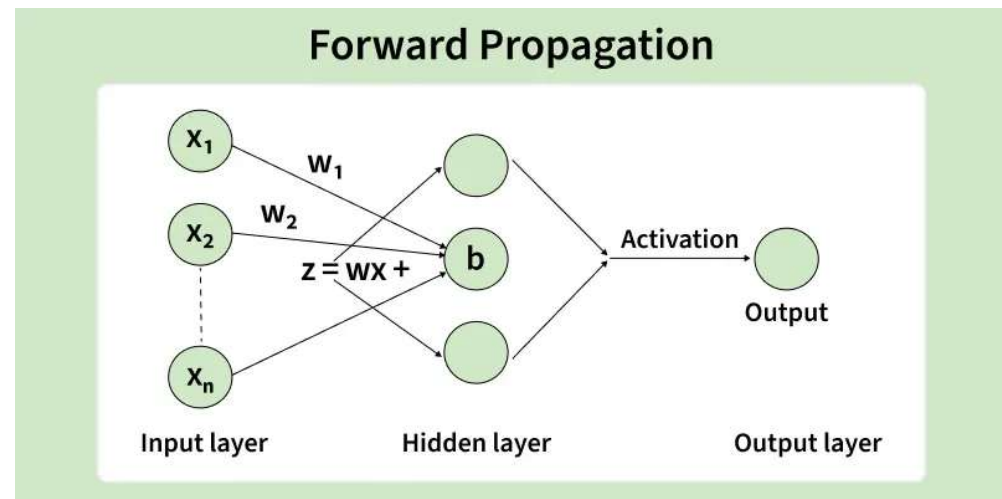
Working of Neural Networks Mechanisms:

- Forward Propagation
- Backpropagation
- Iteration



Forward Propagation

- When data is input into the network, it passes through the network in the forward direction, from the input layer through the hidden layers to the output layer. This process is known as forward propagation.



Forward Propagation

Here's what happens during this phase:

1. Linear Transformation: Each neuron in a layer receives inputs which are multiplied by the weights associated with the connections. These products are summed together and a bias is added to the sum. This can be represented mathematically as:

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

where

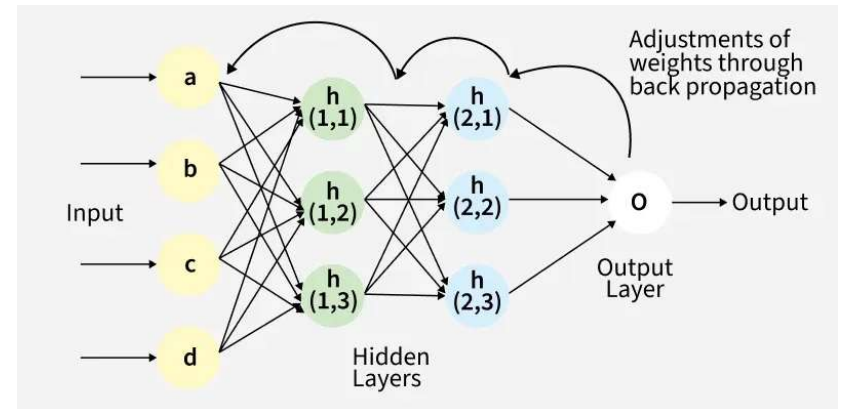
- w represents the weights
- x represents the inputs
- b is the bias

2. Activation: The result of the linear transformation (denoted as z) is then passed through an activation function. **The activation function is crucial because it introduces non-linearity into the system, enabling the network to learn more complex patterns. Popular activation functions include ReLU, sigmoid and tanh.**



Backpropagation

- Backpropagation, short for Backward Propagation of Errors, is a key algorithm used to train neural networks by minimizing the **difference between predicted and actual outputs**. It works by propagating errors backward through the network, using the chain rule of calculus to compute gradients and then iteratively updating the weights and biases. **Combined with optimization techniques like gradient descent, backpropagation enables the model to reduce loss across epochs and effectively learn complex patterns from data.**



Backpropagation

- After forward propagation, the network evaluates its performance using a loss function which measures **the difference between the actual output and the predicted output**. The goal of training is to minimize this loss. This is where backpropagation comes into play:
- **Loss Calculation:** The network calculates the loss which provides a measure of error in the predictions. The loss function could vary; common choices are mean squared error for regression tasks or cross-entropy loss for classification.
- **Gradient Calculation:** The network computes the gradients of the loss function with respect to each weight and bias in the network. This involves applying the chain rule of calculus to find out how much each part of the output error can be attributed to each weight and bias.
- **Weight Update:** Once the gradients are calculated, the weights and biases are updated using an optimization algorithm like stochastic gradient descent (SGD). The weights are adjusted in the opposite direction of the gradient to minimize the loss. The size of the step taken in each update is determined by the learning rate.



Iteration

- This process of forward propagation, loss calculation, backpropagation and weight update is repeated for many iterations over the dataset. Over time, this iterative process reduces the loss and the network's predictions become more accurate.



Example of Email Classification

Email ID	Email Content	Sender	Subject Line	Label
1	"Get free gift cards now!"	spam@example.com	"Exclusive Offer"	1

To classify this email, we will create a feature vector based on the analysis of keywords such as "free" "win" and "offer"

The feature vector of the record can be presented as:

- "free": Present (1)
- "win": Absent (0)
- "offer": Present (1)



How Neurons Process Data in a Neural Network

- In a neural network, input data is passed through multiple layers, including one or more hidden layers. Each neuron in these hidden layers performs several operations, transforming the input into a usable output.

1. Input Layer: The input layer contains 3 nodes that indicates the presence of each keyword.

2. Hidden Layer: The input vector is passed through the hidden layer. Each neuron in the hidden layer performs two primary operations: a weighted sum followed by an activation function.

- Weights:**

- Neuron H1: [0.5, -0.2, 0.3]

- Neuron H2: [0.4, 0.1, -0.5]

- **Input Vector:** [1, 0, 1]

- **Weighted Sum Calculation**

- **For H1:** $(1 \times 0.5) + (0 \times -0.2) + (1 \times 0.3) = 0.5 + 0 + 0.3 = 0.8$

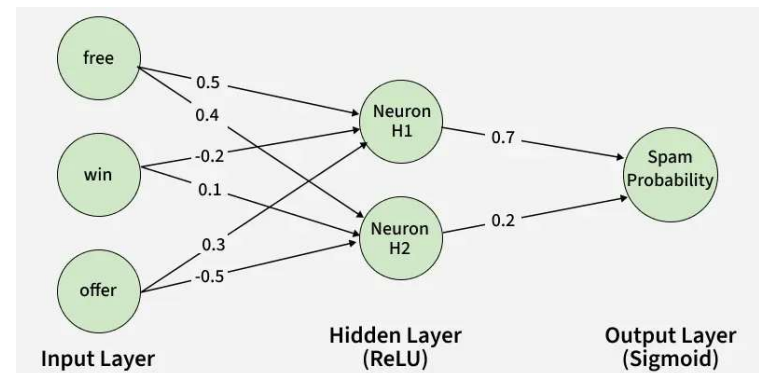
- **For H2:** $(1 \times 0.4) + (0 \times 0.1) + (1 \times -0.5) = 0.4 + 0 - 0.5 = -0.1$

- **Activation Function**

- Here we will use [ReLU activation function](#):

- **H1 Output:** $\text{ReLU}(0.8) = 0.8$

- **H2 Output:** $\text{ReLU}(-0.1) = 0$

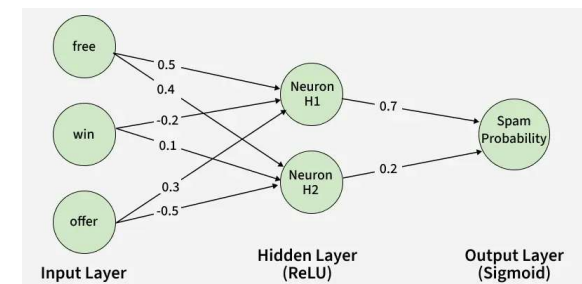


3. Output Layer: The activated values from the hidden neurons are sent to the output neuron where they are again processed using a weighted sum and an activation function.

- **Output Weights:** [0.7, 0.2]
- **Input from Hidden Layer:** [0.8, 0]
- **Weighted Sum:** $(0.8 \times 0.7) + (0 \times 0.2) = 0.56 + 0 = 0.56$
- **Activation (Sigmoid):** $\sigma(0.56) = \frac{1}{1 + e^{-0.56}} \approx 0.636$

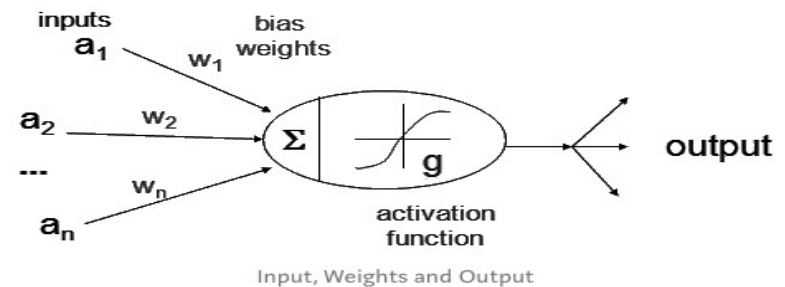
4. Final Classification:

- The output value of approximately **0.636** indicates the probability of the email being spam.
- Since this value is greater than 0.5, the neural network classifies the email as spam (1).



Activation Function

- An activation function in a neural network is a **mathematical function applied to the output of a neuron. It introduces non-linearity, enabling the model to learn and represent complex data patterns.** Without it, even a deep neural network would behave like a simple linear regression model.
- Activation functions decide whether a neuron should be activated **based on the weighted sum of inputs and a bias term.** They also make backpropagation possible by providing gradients for weight updates..



Activation Function

Activation functions are generally two types, These are

- Linear or Identity Activation Function
- Non-Linear Activation Function.
- Generally, neural networks use **non-linear activation functions**.

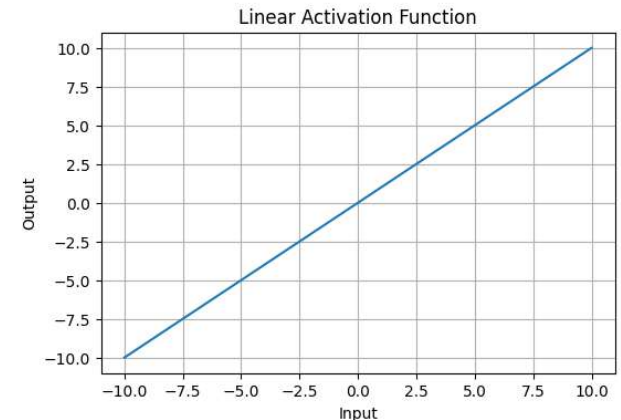
Why Non-Linearity is Important

- Real-world data is rarely linearly separable.
- Non-linear functions allow neural networks to form **curved decision boundaries**, making them capable of handling complex patterns (e.g., classifying apples vs. bananas under varying colors and shapes).
- They ensure networks can model advanced problems like image recognition, NLP and speech processing.



Linear Activation Function

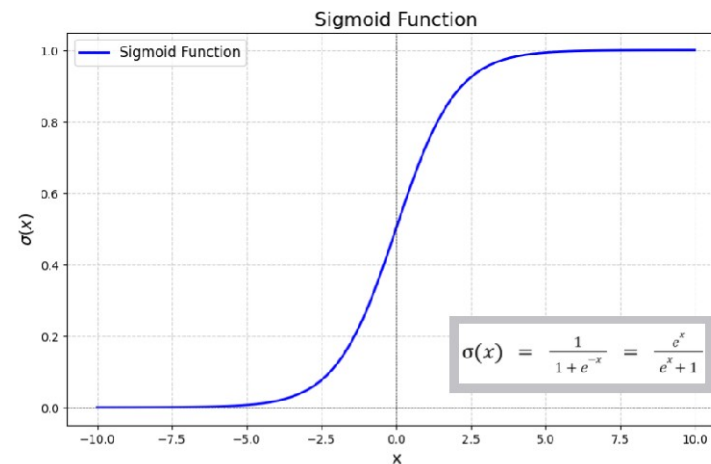
- Linear Activation Function resembles straight line define by $y=x$. No matter how many layers the neural network contains if they all use linear activation functions the output is a linear combination of the input.
- **The range of the output spans from $(-\infty \text{ to } +\infty)(-\infty \text{ to } +\infty)$.**
- Linear activation function is used at just one place i.e. output layer.
- Using linear activation across all layers makes the network's ability to learn complex patterns limited.



Sigmoid Activation Function

Sigmoid Activation Function is characterized by 'S' shape. It is mathematically defined as $\sigma(x) = \frac{1}{1+e^{-x}}$. This formula ensures a smooth and continuous output that is essential for gradient-based optimization methods.

- It allows neural networks to handle and model complex patterns that linear equations cannot.
- The output ranges between 0 and 1, hence useful for binary classification.
- The function exhibits a steep gradient when x values are between -2 and 2. This sensitivity means that small changes in input x can cause significant changes in output y which is critical during the training process.



Tanh Activation Function

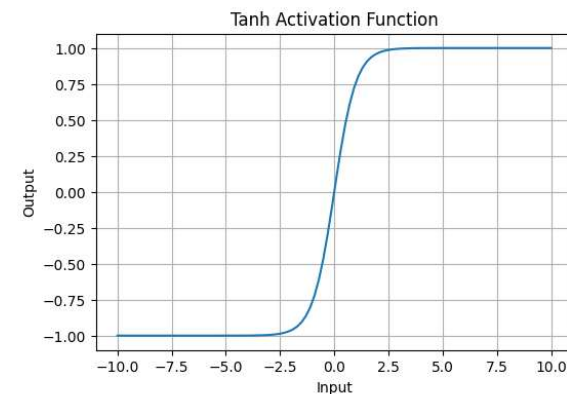
Tanh function (hyperbolic tangent function) is a shifted version of the sigmoid, allowing it to stretch across the y-axis. It is defined as:

$$f(x) = \tanh(x) = \frac{2}{1+e^{-2x}} - 1.$$

Alternatively, it can be expressed using the sigmoid function:

$$\tanh(x) = 2 \times \text{sigmoid}(2x) - 1$$

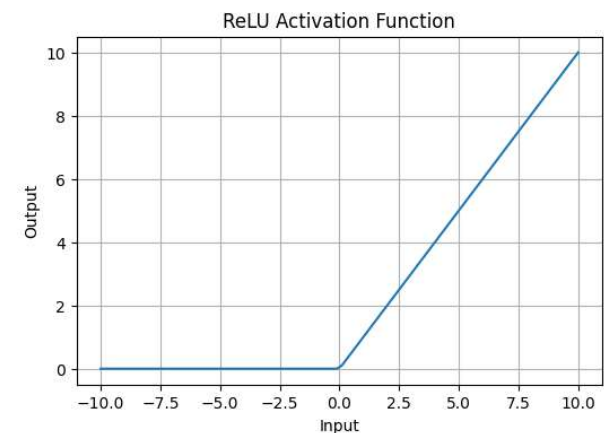
- **Value Range:** Outputs values from -1 to +1.
- **Non-linear:** Enables modeling of complex data patterns.
- **Use in Hidden Layers:** Commonly used in hidden layers due to its zero-centered output, facilitating easier learning for subsequent layers.



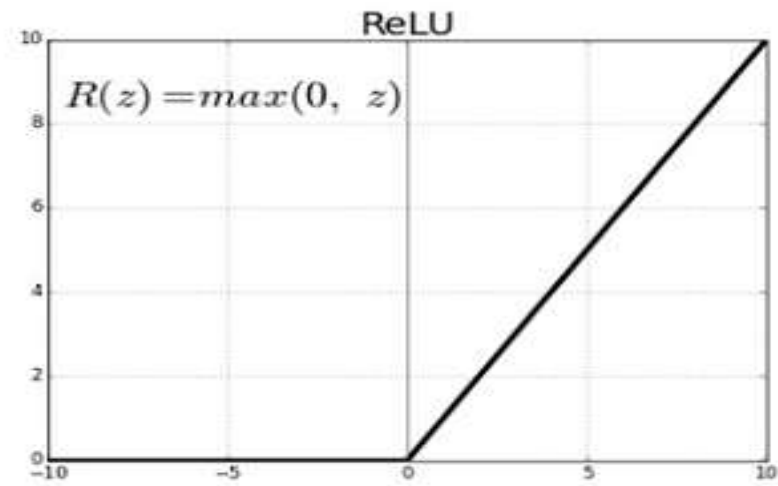
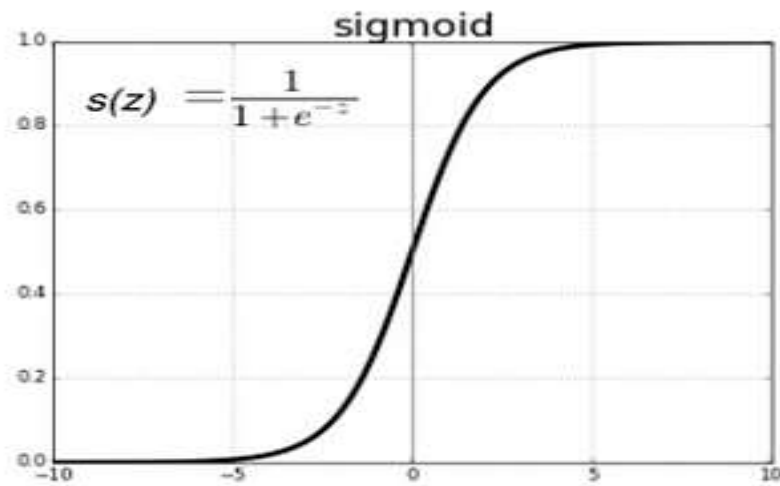
ReLU (Rectified Linear Unit) Function

ReLU activation is defined by $A(x) = \max(0, x)$, this means that if the input x is positive, ReLU returns x , if the input is negative, it returns 0.

- **Value Range:** $[0, \infty)$, meaning the function only outputs non-negative values.
- **Nature:** It is a non-linear activation function, allowing neural networks to learn complex patterns and making backpropagation more efficient.
- **Advantage over other Activation:** ReLU is less computationally expensive than tanh and sigmoid because it involves simpler mathematical operations. At a time only a few neurons are activated making the network sparse making it efficient and easy for computation.



Sigmoid Vs ReLU



Sigmoid Vs ReLU



Comparison of Activation functions

Features	Sigmoid	tanh	ReLU
Range	0 to 1	-1 to 1	0 to ∞
Gradient Issues	Severe vanishing gradients	Moderate vanishing gradients	No vanishing gradient for $x > 0$
Computation Cost	Expensive (exponential)	Expensive (exponential)	Very cheap (max operation)
Use Case	Binary classification outputs	Hidden layers zero-centered data	Deep neural networks
Pros	Smooth, probabilistic output	Better gradient flow than sigmoid	Fast, efficient, avoids vanishing gradient
Cons	Not zero-centered, slow training	Still saturates for large inputs	Dying ReLU problem
Output	Always positive	Symmetric around zero	Sparse

vanishing gradients: Gradients become extremely small, causing weights in early layers to stop updating, leading to "dead" neurons.



Gradient descent and Backpropagation algorithms

- Backpropagation and gradient descent are fundamental algorithms used together to train neural networks by minimizing prediction error.
- Backpropagation calculates the gradient of the loss function with respect to each weight using the chain rule, while gradient descent updates weights in the opposite direction of this gradient to minimize loss iteratively.
- **Gradient Descent (Optimization):** It updates the network's weights and biases using the gradients computed by backpropagation, aiming for a lower error in the next iteration.
- The update rule is $\text{new weight} = \text{old weight} - (\text{learning rate} \times \text{gradient})$.

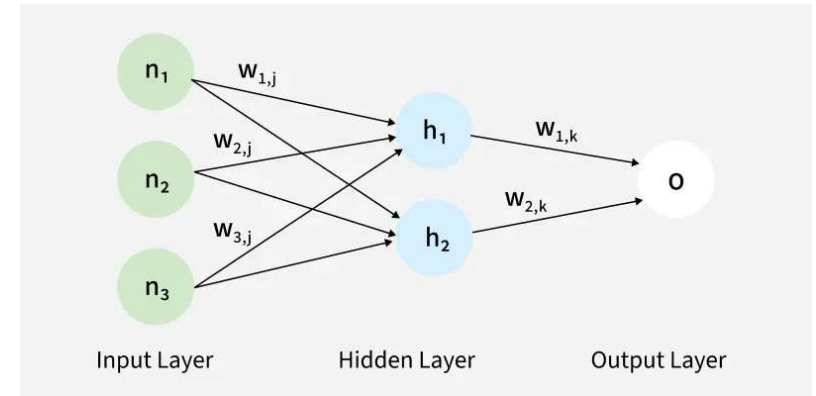


Working of Back Propagation Algorithm

- The Back Propagation algorithm involves two main steps: the Forward Pass and the Backward Pass.

1. Forward Pass

- In forward pass the input data is fed into the input layer.
- These inputs combined with their respective weights are passed to hidden layers.
- For example in a network with two hidden layers (h1 and h2) the output from h1 serves as the input to h2.
- Before applying an activation function, a bias is added to the weighted inputs.
- Each hidden layer computes the weighted sum ($\sum$) of the inputs then applies an activation function like ReLU (Rectified Linear Unit) to obtain the output (σ).
- The output is passed to the next layer where an activation function such as softmax converts the weighted outputs into probabilities for classification.



2. Backward Pass:

In the backward pass the error (the difference between the predicted and actual output) is propagated back through the network to adjust the weights and biases. One common method for error calculation is the [Mean Squared Error \(MSE\)](#) given by:

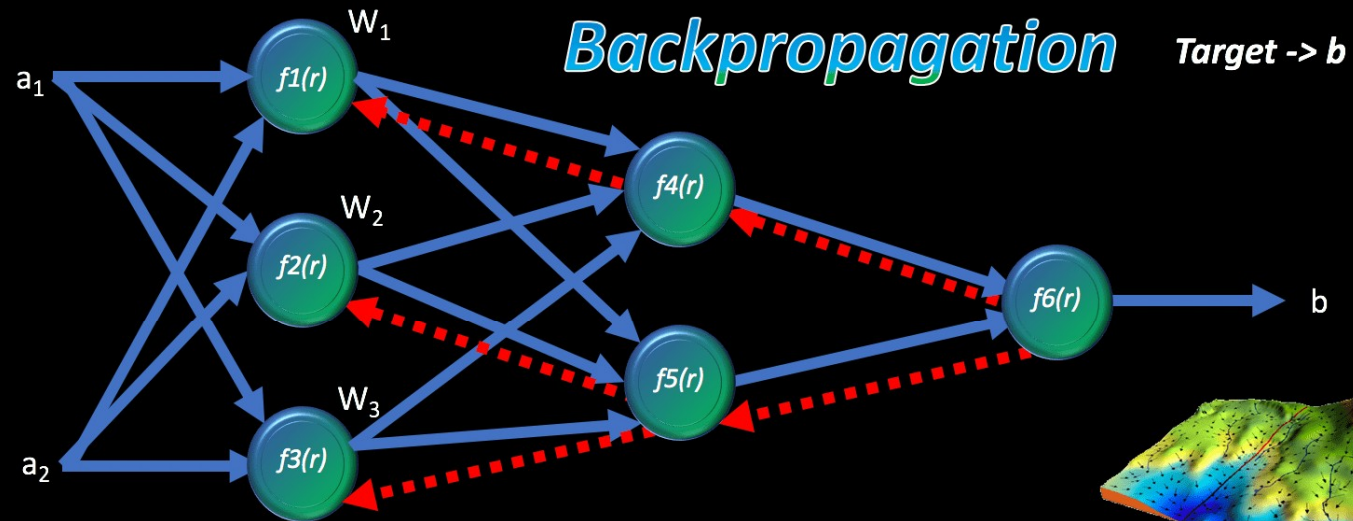
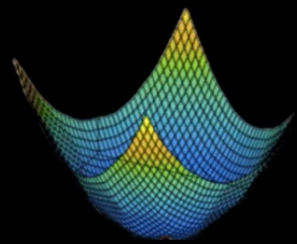
$$MSE = (\text{Predicted Output} - \text{Actual Output})^2$$



Algorithm 1 Backpropagation Algorithm

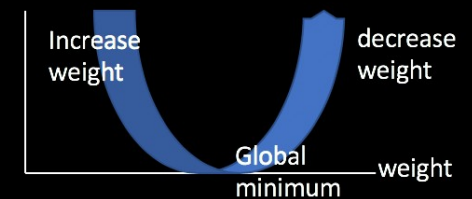
```
1: procedure TRAIN
2:    $X \leftarrow$  Training Data Set of size  $m \times n$ 
3:    $y \leftarrow$  Labels for records in  $X$ 
4:    $w \leftarrow$  The weights for respective layers
5:    $l \leftarrow$  The number of layers in the neural network,  $1 \dots L$ 
6:    $D_{ij}^{(l)} \leftarrow$  The error for all  $l, i, j$ 
7:    $t_{ij}^{(l)} \leftarrow 0$ . For all  $l, i, j$ 
8:   For  $i = 1$  to  $m$ 
9:      $a^l \leftarrow \text{feedforward}(x^{(i)}, w)$ 
10:     $d^l \leftarrow a(L) - y(i)$ 
11:     $t_{ij}^{(l)} \leftarrow t_{ij}^{(l)} + a_j^{(l)} \cdot t_i^{l+1}$ 
12:    if  $j \neq 0$  then
13:       $D_{ij}^{(l)} \leftarrow \frac{1}{m} t_{ij}^{(l)} + \lambda w_{ij}^{(l)}$ 
14:    else
15:       $D_{ij}^{(l)} \leftarrow \frac{1}{m} t_{ij}^{(l)}$ 
16:      where  $\frac{\partial}{\partial w_{ij}^{(l)}} J(w) = D_{ij}^{(l)}$ 
```





Steps:

1. Provides training data $a = [a_1, \dots, a_2, \dots, a_x]$
2. Propagate forward i.e. $b_1 = f_1(W_{(x1)1} \cdot X_1 + W_{(x2)1} \cdot X_2)$, $b_2 = f_2(W_{(x1)2} \cdot X_1 + W_{(x2)2} \cdot X_2)$, $b_3 = f_3(W_{(x1)3} \cdot X_1 + W_{(x2)3} \cdot X_2)$, $b_4 = f_4(W_{14} b_1 + W_{24} b_2 + W_{34} b_3)$, $b_5 = f_5(W_{15} b_1 + W_{25} b_2 + W_{35} b_3)$ and finally signal throughput $b = f_6(W_{46} b_4 + W_{56} b_5)$
3. Next step in algorithm to compare 'b' with the desired output value, calculate difference as error signal 'e' from output layer neuron.
4. So with prediction 'b' now error 'e' is now known for backpropagation
5. Backpropagate error each unit 'r' in each layer end to start
6. Repeat 2 to 5 again till we achieve our goal



<https://AILabPage.com>



Forward Propagation

1. Initial Calculation

The weighted sum at each node is calculated using:

$$a_j = \sum (w_{i,j} * x_i)$$

Where,

- a_j is the weighted sum of all the inputs and weights at each node
- $w_{i,j}$ represents the weights between the i^{th} input and the j^{th} neuron
- x_i represents the value of the i^{th} input

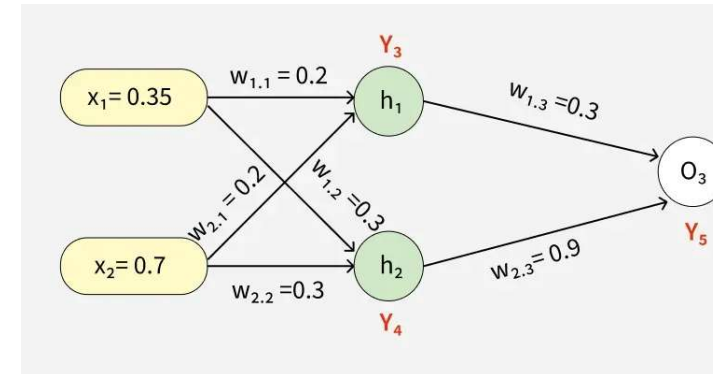
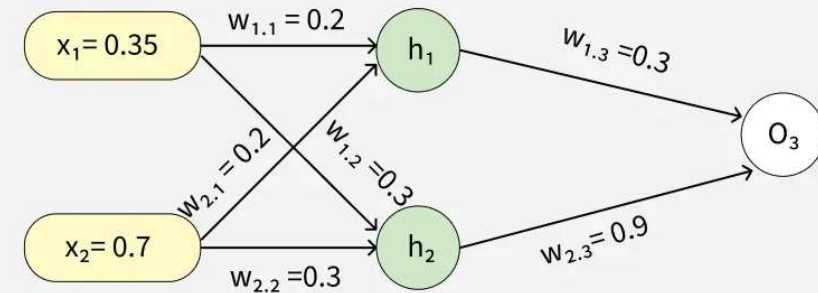
O (output): After applying the activation function to a , we get the output of the neuron:

$$o_j = \text{activation function}(a_j)$$

2. Sigmoid Function

The sigmoid function returns a value between 0 and 1, introducing non-linearity into the model.

$$y_j = \frac{1}{1+e^{-a_j}}$$



3. Computing Outputs

At h1 node

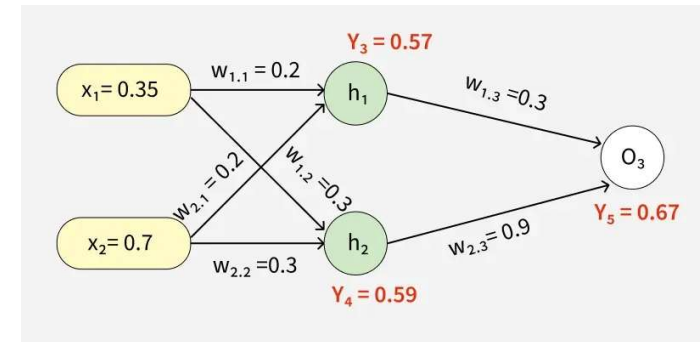
$$\begin{aligned}a_1 &= (w_{1,1}x_1) + (w_{2,1}x_2) \\&= (0.2 * 0.35) + (0.2 * 0.7) \\&= 0.21\end{aligned}$$

Once we calculated the a_1 value, we can now proceed to find the y_3 value:

$$\begin{aligned}y_j &= F(a_j) = \frac{1}{1+e^{-a_1}} \\y_3 &= F(0.21) = \frac{1}{1+e^{-0.21}} \\y_3 &= 0.56\end{aligned}$$

Similarly find the values of y_4 at h_2 and y_5 at O_3

$$\begin{aligned}a_2 &= (w_{1,2} * x_1) + (w_{2,2} * x_2) = (0.3 * 0.35) + (0.3 * 0.7) = 0.315 \\y_4 &= F(0.315) = \frac{1}{1+e^{-0.315}} \\a_3 &= (w_{1,3} * y_3) + (w_{2,3} * y_4) = (0.3 * 0.57) + (0.9 * 0.59) = 0.702 \\y_5 &= F(0.702) = \frac{1}{1+e^{-0.702}} = 0.67\end{aligned}$$



4. Error Calculation

Our actual output is 0.5 but we obtained 0.67. To calculate the error we can use the below formula:

$$\begin{aligned}Error_j &= y_{target} - y_5 \\&=> 0.5 - 0.67 = -0.17\end{aligned}$$



Back Propagation

1. Calculating Gradients

The change in each weight is calculated as:

$$\Delta w_{ij} = \eta \times \delta_j \times O_j$$

Where:

- δ_j is the error term for each unit,
- η is the learning rate.

2. Output Unit Error

For O3:

$$\begin{aligned}\delta_5 &= y_5(1 - y_5)(y_{target} - y_5) \\ &= 0.67(1 - 0.67)(-0.17) = -0.0376\end{aligned}$$

3. Hidden Unit Error

For h1:

$$\begin{aligned}\delta_3 &= y_3(1 - y_3)(w_{1,3} \times \delta_5) \\ &= 0.56(1 - 0.56)(0.3 \times -0.0376) = -0.0027\end{aligned}$$

For h2:

$$\begin{aligned}\delta_4 &= y_4(1 - y_4)(w_{2,3} \times \delta_5) \\ &= 0.59(1 - 0.59)(0.9 \times -0.0376) = -0.00819\end{aligned}$$

4. Weight Updates

For the weights from hidden to output layer:

$$\Delta w_{2,3} = 1 \times (-0.0376) \times 0.59 = -0.022184$$

New weight:

$$w_{2,3}(\text{new}) = -0.022184 + 0.9 = 0.877816$$

New weight:

$$w_{2,3}(\text{new}) = -0.022184 + 0.9 = 0.877816$$

For weights from input to hidden layer:

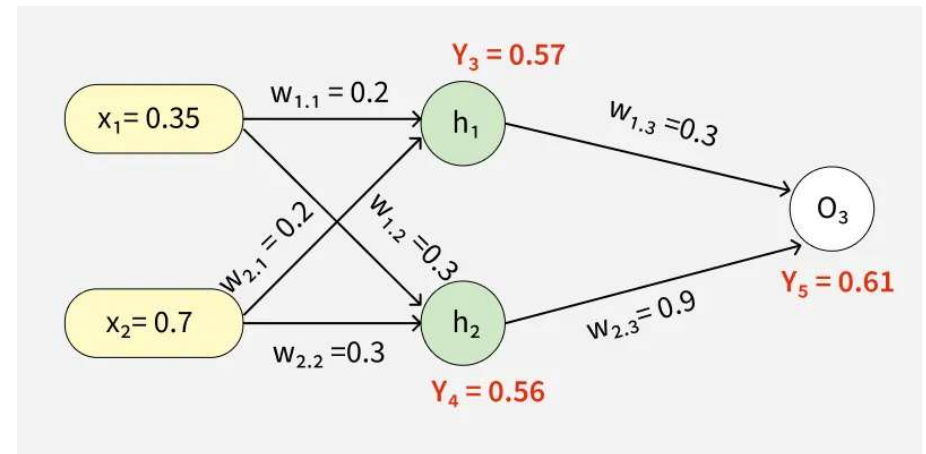
$$\Delta w_{1,1} = 1 \times (-0.0027) \times 0.35 = -0.000945$$

New weight:

$$w_{1,1}(\text{new}) = -0.000945 + 0.2 = 0.199055$$

Similarly other weights are updated:

- $w_{1,2}(\text{new}) = 0.273225$
- $w_{1,3}(\text{new}) = 0.086615$
- $w_{2,1}(\text{new}) = 0.269445$
- $w_{2,2}(\text{new}) = 0.18534$



After updating the weights the forward pass is repeated hence giving:

- $y_3 = 0.57$
- $y_4 = 0.56$
- $y_5 = 0.61$

Since $y_5 = 0.61$ is still not the target output the process of calculating the error and backpropagating continues until the desired output is reached.

This process demonstrates how Back Propagation iteratively updates weights by minimizing errors until the network accurately predicts the output.

$$\begin{aligned} \text{Error} &= y_{\text{target}} - y_5 \\ &= 0.5 - 0.61 = -0.11 \end{aligned}$$

This process is said to be continued until the actual output is gained by the neural network.



NN Simulators for practice

- <https://medium.com/@karna.sujan52/back-propagation-algorithm-numerical-solved-f60c6986b643>
- <https://share.google/bLyCfaeSy7oxu3SGI>
- <https://shorturl.at/PaTdR>
- <https://developers.google.com/machine-learning/crash-course/neural-networks/nodes-hidden-layers>
- <https://www.mladdict.com/neural-network-simulator>
- <https://wandb.ai/mostafaibrahim17/ml-articles/reports/Decoding-Backpropagation-and-Its-Role-in-Neural-Network-Learning--Vmlldzo3MTY5MzY1>

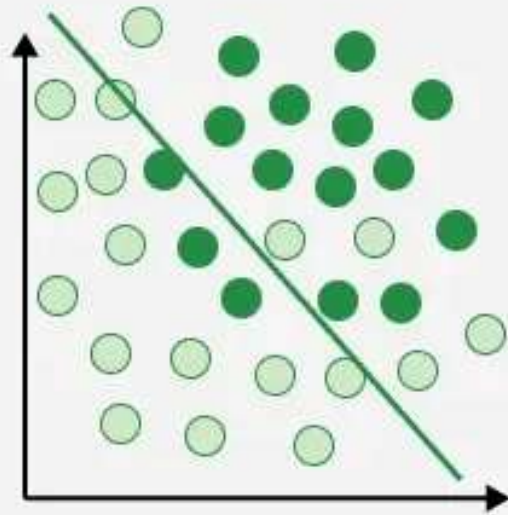


Regularization Techniques in Neural Networks

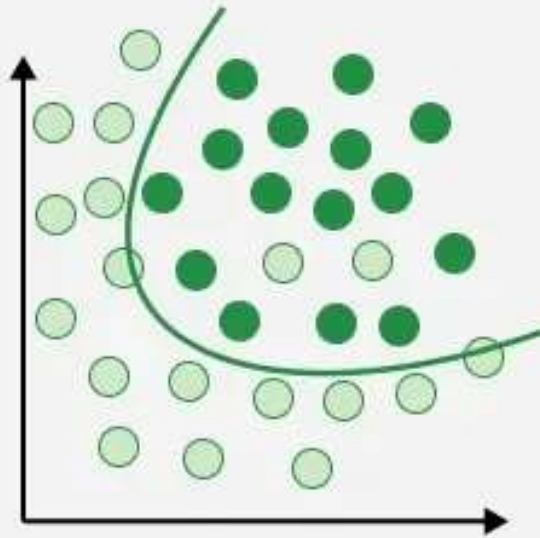
What is Regularization?

- Regularization is a technique used in machine learning **to prevent overfitting, which otherwise causes models to perform poorly on unseen data.** By adding a penalty for complexity, regularization encourages simpler and more generalizable models.
- **Prevents overfitting:** Adds constraints to the model to reduce the risk of memorizing noise in the training data.
- **Improves generalization:** Encourages simpler models that perform better on new, unseen data.

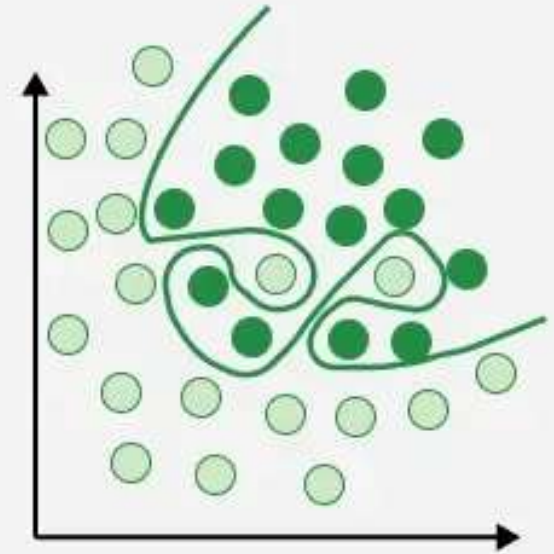




Under-fitting



Appropriate-fitting



Over-fitting

Types of Regularization

- There are mainly 3 types of regularization techniques, each applying penalties in different ways to control model complexity and improve generalization.
- **1. Lasso Regression**
- **2. Ridge Regression**
- **3. Elastic Net Regression**



1. Lasso Regression

- A regression model which uses the L1 Regularization technique is called [LASSO \(Least Absolute Shrinkage and Selection Operator\)](#) regression. It adds the absolute value of magnitude of the coefficient as a penalty term to the loss function(L). This penalty can shrink some coefficients to zero which helps in selecting only the important features and ignoring the less important ones.

$$\text{Cost} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{i=1}^m |w_i|$$

Where

- m : Number of Features
- n : Number of Examples
- y_i : Actual Target Value
- $\hat{y}_i$: Predicted Target Value



Example

- `X, y = make_regression(n_samples=100, n_features=5, noise=0.1, random_state=42)`: Generates a regression dataset with 100 samples, 5 features and some noise.
- `X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)`: Splits the data into 80% training and 20% testing sets.
- `lasso = Lasso(alpha=0.1)`: Creates a Lasso regression model with regularization strength alpha set to 0.1.

```
from sklearn.linear_model import Lasso
from sklearn.model_selection import train_test_split
from sklearn.datasets import make_regression
from sklearn.metrics import mean_squared_error

X, y = make_regression(n_samples=100, n_features=5, noise=0.1, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

lasso = Lasso(alpha=0.1)
lasso.fit(X_train, y_train)

y_pred = lasso.predict(X_test)

mse = mean_squared_error(y_test, y_pred)
print(f"Mean Squared Error: {mse}")

print("Coefficients:", lasso.coef_)
```

Output:

```
Mean Squared Error: 0.06362439921332456
Coefficients: [60.50305581  98.52475354  64.3929265   56.96061238  35.52928502]
```

Lasso Regression

The output shows the model's prediction error and the importance of features with some coefficients reduced to zero due to L1 regularization.

2 Ridge Regression

- Regression model that uses the L2 regularization technique is called [Ridge regression](#). It adds the squared magnitude of the coefficient as a penalty term to the loss function(L). It handles multicollinearity by shrinking the coefficients of correlated features instead of eliminating them.

$$\text{Cost} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{i=1}^m w_i^2$$

Where,

- n : Number of examples or data points
- m : Number of features i.e predictor variables
- y_i : Actual target value for the i th example
- $\hat{y}_i$: Predicted target value for the i th example
- w_i : Coefficients of the features
- λ : Regularization parameter that controls the strength of regularization



Example

- `ridge = Ridge(alpha=1.0)`: Creates a Ridge regression model with regularization strength `alpha` set to 1.0.

```
from sklearn.linear_model import Ridge
from sklearn.datasets import make_regression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error

X, y = make_regression(n_samples=100, n_features=5, noise=0.1, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

ridge = Ridge(alpha=1.0)
ridge.fit(X_train, y_train)
y_pred = ridge.predict(X_test)

mse = mean_squared_error(y_test, y_pred)
print("Mean Squared Error:", mse)
print("Coefficients:", ridge.coef_)
```

Output:

```
Mean Squared Error: 4.114050771972589
Coefficients: [59.87954432  97.15091098  63.24364738  56.31999433  35.34591136]
```

Ridge Regression

The output shows the MSE showing model performance. Lower MSE means better accuracy. The coefficients reflect the regularized feature weights.



3 Elastic Net Regression

[Elastic Net Regression](#) is a combination of both L1 as well as L2 regularization. That shows that we add the absolute norm of the weights as well as the squared measure of the weights. With the help of an extra hyperparameter that controls the ratio of the L1 and L2 regularization.

$$\text{Cost} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \left((1 - \alpha) \sum_{i=1}^m |w_i| + \alpha \sum_{i=1}^m w_i^2 \right)$$

Where

- n : Number of examples (data points)
- m : Number of features (predictor variables)
- y_i : Actual target value for the i^{th} example
- $\hat{y}_i$: Predicted target value for the i^{th} example
- w_i : Coefficients of the features
- λ : Regularization parameter that controls the strength of regularization
- α : Mixing parameter where $0 \leq \alpha \leq 1$ and $\alpha = 1$ corresponds to Lasso (L_1) regularization, $\alpha = 0$ corresponds to Ridge (L_2) regularization and Values between 0 and 1 provide a balance of both L1 and L2 regularization



Example

- **model = ElasticNet(alpha=1.0, l1_ratio=0.5)** : Creates an Elastic Net model with regularization strength $\alpha=1.0$ and L1/L2 mixing ratio 0.5.

```
from sklearn.linear_model import ElasticNet
from sklearn.datasets import make_regression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error

X, y = make_regression(n_samples=100, n_features=10, noise=0.1, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = ElasticNet(alpha=1.0, l1_ratio=0.5)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
mse = mean_squared_error(y_test, y_pred)

print("Mean Squared Error:", mse)
print("Coefficients:", model.coef_)
```

Output:

```
Mean Squared Error: 7785.886176938014
Coefficients: [16.84528938 31.77080959  4.05901996 40.18486737 57.25856154 45.81463318
 58.97979422 -0.          3.82816854 41.1096051 ]
```

Elastic Net Regression

The output shows MSE which measures how far off predictions are from actual values (lower is better) and coefficients show feature importance.



Advantages Regularization

- **Prevents Overfitting:** Regularization helps models focus on underlying patterns instead of memorizing noise in the training data.
- **Enhances Performance:** Prevents excessive weighting of outliers or irrelevant features helps in improving overall model accuracy.
- **Stabilizes Models:** Reduces sensitivity to minor data changes which ensures consistency across different data subsets.
- **Prevents Complexity:** Keeps model from becoming too complex which is important for limited or noisy data.
- **Handles Multicollinearity:** Reduces the magnitudes of correlated coefficients helps in improving model stability.
- **Promotes Consistency:** Ensures reliable performance across different datasets which reduces the risk of large performance shifts.

